TheoryBERT: BERT Language Analysis with Verified Formal Representations

Abstract - TheoryBERT is a local web system for translating English descriptions into formal-language objects and inspecting their behavior. BERT supplies language analysis; exact algorithms validate automata, derivations and arithmetic intermediate representations. This paper documents a direct DFA predictor, a separately trained grammar decoder and a new typed semantic-plan decoder with learned literal copying. The plan experiment trains a compact head over immutable BERT features, preserves the existing visualization and uses language-grouped supervision. Evaluation distinguishes first-candidate equivalence, bounded search success, grammar production accuracy and evidence validity. The new head improves first-candidate equivalence from 87.68% to 95.32% on a 641-case regression set and from 7.02% to 66.67% on 57 matched fresh cases, with slightly slower inference. Remaining wording failures, rejected candidates and dataset lineage delimit the result; a valid diagram is not proof of correct English interpretation.

Index Terms - BERT, deterministic finite automata, semantic parsing, context-free grammar, compiler education, formal verification.

I. INTRODUCTION

A. Problem Statement

Students of theory of computation and compiler construction must connect an informal requirement to a mathematical object, then connect that object to a concrete execution. A request such as accepting binary words ending in 0101 describes an infinite language. A student must infer the necessary remembered suffixes, supply complete transitions, identify accepting states and distinguish the empty input from a rejected nonempty word. A diagram alone leaves these obligations implicit. The difficulty increases for negation, exact counts, modular conditions and combinations of constraints.

The practical problem addressed here is to make those obligations inspectable in one interface. The user describes a language in English and supplies a test string. The system should construct a relevant candidate, show its actual states and transitions, simulate that particular input and expose the evidence supporting any verification claim. For grammars, it should generate productions, parse a supplied token sequence and display a production-checked derivation. For compiler exercises, it should show how the accepted expression gives rise to an abstract syntax tree and symbolic intermediate code.

The local computer has 8 GB of memory and limited storage. Training bounds caches and avoids simultaneous heavy jobs. Efficiency includes compute and storage alongside accuracy, with an explicit checkpoint record for rollback.

A common implementation failure is a reusable demonstration diagram whose content does not follow the current requirement. Such a diagram can animate smoothly while representing the wrong language. A second failure is to change a DFA whenever the test word changes, even though the language definition has stayed fixed. A DFA represents the language; a trace represents one word. TheoryBERT therefore treats a new language description as a new construction request and an input-only edit as a new simulation of the same machine. Different strings may correctly traverse the same graph in different ways.

Correctness has several distinct meanings. A complete transition table may be structurally valid but semantically wrong. A trace may faithfully follow that wrong table. Equivalence to an independent reference DFA is a stronger, decidable claim, but the reference itself must represent the intended requirement. With arbitrary English alone, there is no independent mathematical oracle. The product must preserve these distinctions instead of labelling every successful prediction as verified. This separation is especially important in an educational interface, where a confident graphic can conceal a misunderstood quantifier or boundary condition.

B. Existing Approaches

Rule-driven interfaces can map a small set of phrases to known automata constructors. They are efficient within their accepted syntax, but extending the vocabulary or adding a new composition often requires another handwritten linguistic condition. The project retains such a baseline for controlled evaluation, while the learned application path does not invoke it. Manual automata and grammar entry remain useful for direct formal exercises; they are labelled as supplied objects and do not count as learned language understanding.

Neural semantic parsing learns mappings from descriptions to executable forms. Earlier natural-language regular-expression work established both semantic-unification and learned translation approaches [1], [2]. A direct neural DFA head has a simple output contract: predict the number of states, accepting-state bits and transition destinations. However, every additional state introduces many coupled decisions. A long literal or a conjunction can fail even when the model has recognized the general language family. Training on more templates can improve familiar wording without resolving this structural burden.

C. Proposed Solution and Objectives

The proposed architecture assigns language interpretation to BERT and formal execution to explicit mathematical components. The direct model is retained as a measured baseline. The new model predicts a short typed language plan, such as ENDS(0101) or AND(CONTAINS(01), COUNT_MOD(1, 2, 0)). The plan is a semantic target learned from English, not a program selected by matching English keywords. Learned start and end pointers copy multi-bit literals from BERT token spans. An exact compiler then constructs and minimizes the corresponding DFA.

This representation gives the neural component fewer dependent graph decisions for common regular-language requirements. It also makes literal identity explicit: the pattern 00101 must keep its leading zeros. Type constraints permit only well-formed operator arguments, while formal bounds prevent uncontrolled products or very large machines. These constraints establish syntactic and structural discipline. They do not guarantee that the predicted operator or numeric argument matches the English request.

The primary objective is improved first-candidate equivalence on frozen fresh wording while preserving established validation behavior. Secondary objectives are fewer trainable parameters, BERT-only learned interpretation, no hidden rule fallback and unchanged visualization. A smaller head need not make inference faster: measured latency includes plan search and construction.

For CFG and compiler modes, BERT predicts grammar-production tokens through a separate decoder over the same evaluated encoder. Earley parsing then checks membership of a particular token sequence. Production checks validate each returned derivation and its yield. The arithmetic compiler lowers an appropriate checked parse to an AST and three-address-style symbolic instructions. These are explicit formal operations over BERT-produced structure. They do not use the automata plan vocabulary to pretend that all context-free languages are regular.

A further objective is scientific traceability. Every generated corpus is tied to its construction code, split counts and a content fingerprint. Equivalent DFA languages are grouped before splitting. Public data filtering is documented. Tests that have been inspected become regression evidence rather than remaining indefinitely described as unseen. Candidate selection is recorded before the fresh test partition is evaluated. These practices make a negative result useful and prevent a polished interface from substituting for model evaluation.

D. Correctness and Educational Representation

A useful educational answer should reveal the model interpretation and its consequences. For example, an exact-word request for 00101 must reject every proper extension even if that extension contains the pattern. A substring request must accept those extensions. The two machines can share visual conventions while differing in their state meanings and accepting behavior. Such minimal semantic differences should be evaluated directly rather than hidden inside a broad average score.

The same principle applies to a numeric requirement. Exactly two ones, at most two ones and an even number of ones are different languages, despite shared words and symbols. An interpretation error may produce a complete machine with no malformed transitions. Therefore the final output needs both an inspectable object and task-appropriate semantic evidence. A displayed sample of accepted words is helpful, but agreement on a finite sample cannot replace exact equivalence when a reference DFA is available.

The system also distinguishes a language specification from an execution specimen. The graph is chosen by the former; the highlighted route is chosen by the latter. This avoids teaching the misconception that each character in an input should become a new DFA state. In grammar mode, however, a longer word may require a larger derivation tree while the productions remain fixed. The application must preserve that mathematical difference instead of forcing every task into one visual template.

The proposed work consequently treats interpretation quality, formal correctness, representation fidelity and resource cost as separate objectives. An improvement is useful only if its measured semantic gains survive validation and the application continues to display the corresponding object accurately. These requirements determine the experiment and the unchanged rendering contract.

E. Organization of the Paper

Section II reviews related work. Section III presents the architecture. Section IV states contributions, and Section V documents data. Section VI reports experiments. Section VII describes the web system and resources. Sections VIII and IX discuss limitations and conclude. A companion report expands provenance and reproducibility.

II. LITERATURE REVIEW

A. Language Representations and Domain Adaptation

Vaswani et al. introduced the attention-based Transformer architecture [3]. BERT supplies the bidirectional representation foundation used here [4]. RoBERTa examines how pretraining choices and data affect comparison [5], motivating caution about unmatched conditions. TheoryBERT retains its pinned BERT encoder rather than silently substituting RoBERTa. Its downstream head must still learn a formal interpretation; attention alone does not specify automata states or grammar productions.

Tai et al. study constrained-resource vocabulary extension through exBERT [6]. BioBERT provides a domain-pretraining example in biomedicine [7], while Weber et al. describe token-augmented fine-tuning [8]. These support studying adaptation choices, not assuming every extension improves correctness. TheoryBERT is exBERT-inspired, with a residual module after the backbone rather than an exact reproduction. Its direct V3 model updates two BERT blocks; the new plan experiment freezes that evaluated encoder.

For this project, the relevant research question is operational: can a smaller, semantically explicit target improve language interpretation without updating the large encoder again? The answer depends on the existing encoder, available supervision and target scope. Added domain tokens may change tokenization, but reduced token fertility is not proof of better semantics. The repository records multiple embedding-initialization choices, including a deliberately large plus-one shift. Their full matched-capacity, multi-seed accuracy ablation has not been executed, so it cannot support an improvement claim.

B. Natural Language to Regular Expressions

Kushman and Barzilay connect natural-language requests with regular expressions using semantic unification [1]. An important lesson for this implementation is that surface-form equality is inadequate: distinct formal expressions can denote the same language. TheoryBERT uses exact DFA equivalence for its automata metric and canonical language identity for split grouping. Its neural architecture differs, but the evaluation should still respect semantic equivalence rather than rewarding one arbitrary spelling of a regular expression.

Locascio et al. investigate learned translation from natural language to regular expressions and provide a public code/data resource [2], [9]. TheoryBERT imports only a restricted portion of the KB13 mirror, because the local task is binary and its regex compiler deliberately supports a bounded dialect. Published performance on the original task cannot be compared directly with the present filtered, generated benchmark. Dataset reuse is therefore accompanied by revision-pinned links, filtering counts and an explicit distinction between source pairs and retained local examples.

C. Intermediate Representations and Compositionality

Herzig et al. examine how intermediate representations affect compositional generalization in pretrained sequence-to-sequence semantic parsers [10]. Their findings motivate investigating a target representation aligned more closely with a request. The TheoryBERT plan experiment applies that design idea to a different architecture and formal domain: a frozen BERT encoder, a compact parallel decoder and an automata compiler. It is an adaptation of an established research direction, not a reproduction of their system or an assertion of comparable benchmark performance.

The local representation makes the distinction concrete. A suffix requirement can be expressed by one operator and a literal rather than a separately predicted destination for every state and alphabet symbol. A conjunction can preserve two interpretable subexpressions before a product construction combines their languages. This reduces output entanglement for covered families. It also restricts expressiveness: a complete arbitrary transition system may not have a short expression in the chosen vocabulary. Structural reconstruction and typed-plan prediction are related tasks with different coverage.

Oren et al. study contextual representations, alignment supervision and template balance in compositional parsing [11]. Their analysis motivates literal pointers and composition-specific evaluation here. TheoryBERT does not implement all their mechanisms. Its uneven generated operator distribution also prevents an aggregate score from establishing systematic generalization.

A particularly important distinction is between a new wording and a new formal target. Shared templates can make a language-held-out test easier than an unrestricted human-written request. Conversely, a new wrapper can expose phrasing sensitivity without testing a new operator. The frozen v2 comparison contains protected formal language groups and a distinct wording wrapper; it is still generated from the same plan family catalogue. Its purpose is a controlled local comparison, not an unrestricted natural-language benchmark.

D. Constrained Decoding

PICARD constrains autoregressive generation by rejecting tokens that do not satisfy a target-language parser [12]. The connection here is the general principle that formal output constraints should be enforced during decoding. TheoryBERT uses a bounded typed prefix search over a small operator vocabulary, with literal spans restricted to binary text. Its head predicts all output positions in parallel. It does not use PICARD software, its incremental SQL parser or its T5 training protocol.

A constrained decoder can still make semantic errors: ENDS(01) and STARTS(01) are both valid plans. Evaluation measures first-candidate equivalence before reference-assisted selection and reports within-five success separately. A checker-selected answer is not the model's unaided first prediction.

E. Behavioral Testing and Formal Execution

Ribeiro et al. propose CheckList to expose model failures through behavioral testing beyond a single held-out accuracy value [13]. This motivates targeted tests for negation, boundaries, literal substitution and combinations of requirements. TheoryBERT keeps known product fixtures, a retired mixed challenge and a newly frozen comparison. A fixture is a regression obligation once it has influenced development. It cannot also serve as independent confirmation that a newly trained model generalizes.

Classical automata and parsing methods supply the executable semantics after language analysis. Cornell's compiler notes explain total DFA transitions and the connection between regular specifications and lexical analysis [14]. NLTK documents incremental Earley chart parsing [15].

TheoryBERT uses these established principles to validate structures, simulate words and check derivations. The project contribution is the integration and evidence discipline, rather than a new DFA equivalence or general CFG parsing algorithm.

Exact DFA equivalence uses product exploration. General CFG equivalence is not supplied. A parse verifies membership under one grammar, not all-word correspondence with English. Preserving an AST in symbolic intermediate code is likewise a bounded claim, without a full-language optimizer or target-machine execution.

F. Research Gap and Positioning

The local engineering gap is a consistent end-to-end contract between learned interpretation and an educational representation. A system can have a useful neural model but still display a stale graph. It can also have a correct symbolic renderer while silently constructing the language with handwritten English rules. TheoryBERT addresses this gap by passing explicit model-produced objects through formal checks and by drawing the resulting machine or derivation. The rendering system has no separate authority to invent a convenient default graph.

Formal-language research also cautions against extrapolating finite accuracy to unrestricted capability. Deletang et al. empirically study neural generalization across the Chomsky hierarchy [16]. Strobl et al. survey transformer expressivity under differing architectural assumptions [17]. TheoryBERT instead predicts bounded formal specifications, then executes exact algorithms. Its new wording comparison tests that complete local approach, without claiming a reproduction of either theoretical analysis or universal language recognition.

The approach combines established pretrained representations, intermediate semantics, formal constraints and behavioral testing. It claims no first invention, universal correctness or superiority to large language models. Human-authored evaluation, matched-data ablations and several seeds remain necessary.

III. PROPOSED SOLUTION

A. Component Architecture

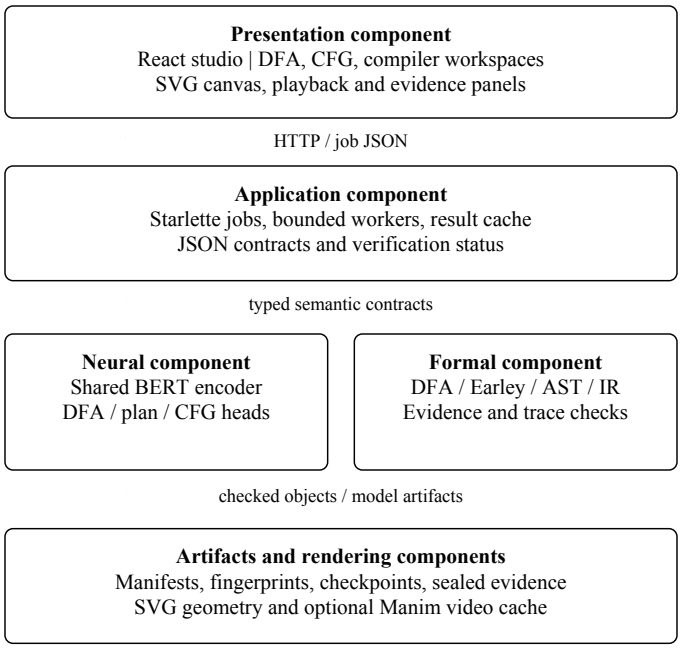


Fig. 1. Component architecture and named formal data interfaces.

The browser owns presentation and playback. Starlette owns model loading, jobs, validation and result evidence. A shared BERT encoder serves the direct DFA predictor and the small grammar or semantic-plan heads. The formal component accepts typed objects, not raw English. It supplies automata construction, simulation, equivalence, CFG parsing and symbolic compiler checks. A separate rendering component consumes these checked structures and their traces. Dataset manifests and model fingerprints provide the artifact boundary.

This organization avoids a second language interpreter in the renderer. Changing an English description invokes the selected BERT predictor. Changing a test word reuses the formal object and computes another trace. The SVG geometry, node styles, edge routing and playback controls remain the same when a different prediction architecture is selected. The neural and formal components share a versioned JSON DFA contract, so an improved interpreter need not redesign the final visual representation.

B. Direct Neural Baseline

The baseline predicts state count, accepting bits and binary transition destinations from a pooled BERT representation. It supports at most eight states. Training masks inactive states and destinations; inference limits destinations to the predicted active range. Bounded alternatives modify uncertain local decisions without consulting a reference. Exact checking then determines whether a candidate matches an independently supplied DFA.

C. BERT Semantic Plan Decoder

The new head projects contextual BERT features from width 768 to 128. Twenty-four learned output queries attend through two Transformer decoder layers, with four attention heads and feed-forward width 256. A token classifier predicts operators, constants and termination. Separate start and end projections score literal boundaries against encoder tokens. A single-bit pattern uses a learned fixed output class; longer patterns use the learned copy action and token-span scores.

The typed vocabulary includes universal and empty languages; prefix, suffix, substring and exact-word requirements; Boolean negation, conjunction and disjunction; count equality and bounds; and count or length modulo conditions. Numbers are restricted to 0 through 16. The compiler accepts binary literals up to 32 characters and at most 64 final states. The generated training catalogue covers patterns up to eight characters and moduli 2 through 8. Implementation capacity must not be confused with demonstrated training coverage.

Training minimizes token cross-entropy plus weighted start/end cross-entropies at copy positions. Padding is excluded. Encoder weights are immutable and fingerprinted. A packed float16 token-feature cache stores real tokens only; batches are padded on demand. The head is selected using validation plan-and-copy exactness, followed by independent DFA validation gates. The test partition is excluded from feature extraction, gradient updates and checkpoint selection.

At inference, a bounded beam tracks the argument types still required by the predicted prefix expression. It ranks up to three token alternatives for a type and up to two literal spans for copying, with beam width eight. Search uses relative neural score gaps and deduplicates completed canonical DFAs. The formal compiler rejects invalid arguments and bounds. There is no English keyword matcher, reference-guided plan construction or replacement of an incorrect prediction with a hand-constructed answer.

D. Formal and Grammar Contracts

Primitive plans compile to exact automata. Negation complements a complete DFA; conjunction and disjunction use reachable product construction. A 256-state intermediate-product limit precedes final minimization. Product BFS compares candidate and reference languages and returns a shortest distinguishing word on disagreement. An empty list is the empty-word witness; a missing witness has a different meaning.

For CFG tasks, the separate BERT head predicts production tokens. Parsing validates each production and terminal yield before a tree is released. Arithmetic lowering verifies temporary definitions, operator arity, operand order and reconstruction of the AST expression. These guarantees concern the predicted formal objects. Correct interpretation of the original English still depends on the learned model and an appropriate evaluation reference.

IV. KEY CONTRIBUTIONS

1) BERT-centered interpretation with explicit semantics. The system provides direct DFA and learned grammar paths and implements a typed plan alternative with learned literal copying. Formal constructors operate on the predicted specification. They do not recognize English requirements independently.

2) Evidence-aware visualization. States, edges, accepting roles, parse nodes and execution highlights derive from the actual formal object and current input. Verified automata, unverified candidate previews and supplied objects retain distinct provenance. The final visual components are preserved across the new model experiment.

3) Auditable local evaluation. Canonical language grouping, dataset fingerprints, per-case outcomes, validation-only promotion gates and a frozen wording comparison provide a traceable record. Failed candidates and small challenge results remain visible. More data is evaluated as a hypothesis, not an automatic improvement.

4) Resource-conscious implementation. Frozen encoder reuse, a small task head, packed cached features, two-thread CPU execution and disk/memory guards enable a controlled experiment on the available machine. Reduced trainable capacity is reported separately from measured inference latency.

These are integration and evaluation contributions. Pretrained encoders, intermediate representations, constrained decoding and classical formal algorithms have prior art. No first-invention or state-of-the-art claim is made.

V. DATASETS AND PROVENANCE

A. External Sources

The downloaded pretrained artifact is google-bert/bert-base-uncased [18], pinned to revision 86b5e0934494bd15c9632b12f734a8a67f723594. BookCorpus and English Wikipedia are inherited pretraining sources described by the model card; the project did not download or retrain on those raw corpora. The source artifact license is Apache-2.0.

The external supervised source is the KB13 description/regex mirror in DeepRegex [9], pinned to revision 096490db7f4b0394fbb46b914cb35a0aa1cba29c. Of 824 paired rows, 55 pass local import restrictions; 497 use unsupported regex dialect, 271 become constant languages after binary restriction and one exceeds a repeat bound. Deduplication leaves 47 public-regex rows in the V3 corpus. The supplied dataset license is MIT. These figures describe this import, not original benchmark coverage.

B. Local Supervision and Evaluation Sets

TABLE I. Local corpus versions. Versions overlap and must not be summed as independent data.

Corpus	Rows	Train / val / test
Shipped sample	261	181 / 39 / 41
English DFA v1	1,785	1,363 / 213 / 209
Expanded DFA	3,207	2,437 / 391 / 379
DFA V3	6,275	4,889 / 745 / 641
Robust DFA candidate	6,738	4,924 / 1,173 / 641
Grammar original	1,299	999 / 170 / 130
Grammar robust	1,652	1,267 / 255 / 130
Semantic plan v1	11,415	9,187 / 1,171 / 1,057
Fresh plan v2	254	0 / 134 / 120

The shipped sample contains 96 unique languages. English DFA v1 and its expanded version contain 208 each. V3 contains 696, including substring, prefix, suffix, modulo, literal, negation, bounded-count and edge cases. The robust direct candidate contains 1,468. The original grammar corpus has 123 distinct grammars; robust merging yields 119 after canonicalization and deduplication. Arithmetic grammars are training/product fixtures, so their preset success is not held-out grammar generalization.

The plan dataset contains 2,914 canonical DFA languages. It annotates 6,274 prior V3 rows with copyable equivalent plans and adds generated supervision; one prior row has no supported copyable annotation. Languages inherit protected prior split roles, and new identities use a deterministic hash split. A retired challenge development subset contributes supervision only when its canonical language belongs to training. Assertions enforce one partition per language.

The older mixed challenge has 165 cases: 100 DFA, 38 CFG and 27 compiler requests, split into 71 development, 46 validation and 48 test cases. It has been inspected and is now regression evidence. The new 254-case comparison is frozen before predictions and contains distinct canonical languages, a separate wording wrapper, 134 validation cases and 120 test cases. All its languages are outside plan training. Matched analysis limits targets to eight states and descriptions to the baseline's 64-token input bound.

Generated local datasets are not claimed to be publicly hosted. The comprehensive report supplies local filenames, hashes, generator commands and public source links. The new comparison fingerprint begins 550b42a0566af360; the complete SHA-256 value is recorded with the corpus. Re-running a frozen comparison requires a new version rather than silently replacing its contents.

VI. EXPERIMENTS AND RESULTS

A. Protocol and Metrics

The principal automata metric is exact equivalence of the first model candidate with an independent reference DFA. Unsupported or invalid predictions count as incorrect. Success within five candidates measures reference-assisted search and is reported separately. Structural validity measures complete well-formed automata; it is not semantic accuracy. The checker releases a sealed verified object only after equivalence. Correctness among released objects is therefore a checker property, not an independent neural achievement.

The new promotion protocol is fixed before prediction. It requires at least a 10 percentage-point first-candidate gain on matched fresh validation cases, at least 88.34% on the original 745-case V3 validation set and success on all known product fixtures. The validation threshold allows a bounded two-point regression from the existing 90.34% value. The selection decision is written before the 120-case fresh test is evaluated. There is no retraining against the resulting test failures.

Both models receive the same comparison descriptions. Matched targets obey the direct model's eight-state limit and 64-token input bound. Additional plan-capacity cases are reported separately. Timing includes tokenization, BERT inference, neural decoding and formal construction. These are local single-process measurements with two CPU threads, not a controlled cross-device speed benchmark. Single-seed model training and synthetic supervision limit the strength of causal claims.

B. Established Results and Failed Expansion

The active V3 direct model achieves 562/641 exact first predictions (87.68%) and 599/641 successes within five (93.45%) on its generated test set. In an earlier matched 379-example comparison, expansion improved first-candidate equivalence from 62.53% to 90.24%. Those scores concern different evaluation sets and must not be concatenated into a single learning curve. A subsequent robust direct fine-tune was rejected because its original validation regression and product failures violated the selection gates.

The mixed reserved challenge showed DFA 7/23 both before and after that rejected expansion. The grammar replacement improved 3/14 to 10/14 on CFG cases, while compiler pipeline success improved 0/9 to 9/9. The compiler subset includes both expected rejection and valid-expression cases. Its size is small, and the figures are not general compiler accuracy. The generated grammar test has 130/130 exact production outputs, but the much weaker fresh-language behavior demonstrates the importance of wording-sensitive evaluation.

C. New Semantic Plan Comparison

TABLE II. Frozen comparison. Selection uses validation only; within-five success uses a reference.

Fresh test metric	Direct V3	Plan head
All cases	120	120
First equivalence	3.33%	71.67%
Within five	5.83%	99.17%
Matched cases	57	57
Matched first	7.02%	66.67%
Original test first	87.68%	95.32%
Fresh median seconds	0.0340	0.0397

The plan head has 537,385 trainable parameters compared with 19,015,056 in the direct V3 training condition. It completes in 230.8 seconds including frozen feature extraction and head training. The best internal validation plan-and-copy exactness is 94.19%. This teacher-target metric differs from decoded DFA equivalence and is not used interchangeably with the table's semantic score.

On matched fresh validation, the observed first-candidate change is 51.35 percentage points. Original validation equivalence for the candidate is 98.39%, and product-fixture success is true. The candidate is promoted to the local default. This decision follows the frozen protocol rather than choosing whichever test result appears most attractive.

The fresh test changes from 4/120 to 86/120 first-candidate equivalent DFAs. These are paired requests, but the training data, target vocabulary and decoder architecture also changed. The experiment therefore evaluates the complete local approach. It does not isolate the effect of intermediate representation alone or establish that freezing BERT is universally preferable to fine-tuning.

Fresh-test operator strata reveal uneven gains: exact words are correct in 23/23 cases and substring requirements in 21/21, while suffix requirements remain only 1/18. Within the matched subset, 35 previously wrong cases become correct, one previously correct case becomes wrong, three stay correct and 18 remain wrong. These small strata expose a substantial remaining wording weakness; the overall gain is not universal reliability.

Per-case records retain formal targets, latency and candidate outcomes. The committed selection and renderer fingerprints make the comparison auditable. API and browser checks confirm that the application loads the selected checkpoint and displays its actual machine.

VII. WEB SYSTEM AND RESOURCE CONTROL

A. Website and Interaction Design

The React/Starlette product provides DFA, CFG and arithmetic workspaces, settings and research information. Reusable SVG components display server-produced formal objects. Python owns the representations and verification decisions; the browser draws their content and controls playback. The project source is identified in [19].

DFA mode accepts an English requirement, a binary word and a prediction source. Learned mode uses the active evaluated checkpoint. Demonstrations, incorrect fixtures and supplied JSON retain distinct provenance. Benchmarks can supply independent references; free-form requests receive structurally checked previews without an equivalence claim.

The graph displays the actual states, total transitions and accepting set. Playback follows the current input trace. Access strings, accepting completions, visits and example words are computed from that machine. Changing only the input changes execution while preserving the language graph.

CFG mode shows predicted productions, parse alternatives and checked derivations. Character, terminal-token and arithmetic-lexeme modes have explicit contracts. The compiler preserves original lexemes in its AST and displays derived symbolic instructions. Syntax failures retain the actual model-produced grammar for inspection.

B. API and Evidence Boundaries

The API supplies health, catalog, research, bounded asynchronous jobs, simulation and cached-video routes. Model caching lets grammar and plan heads share their fingerprinted immutable encoder. The server owns result evidence, provenance and revision invalidation.

Optional Manim export requires sealed DFA equivalence evidence or a production-checked tree. The renderer rechecks evidence and reconstructs traces. An unverified prediction remains an SVG preview with a clearly narrower claim.

C. Preserving the Final Visualization

The canvas, graph layout, Manim scenes, renderer and two principal style sheets remain byte-identical to their pre-experiment hashes. The model dispatcher still supplies the same DFA JSON contract. A different graph therefore follows a different predicted language, without a special display template.

Presentation stays stable while content changes. A longer suffix can require more states; a longer input can revisit existing states without adding nodes. Tests compare visible counts with the formal object and playback with simulation, guarding both graph construction and execution fidelity.

D. Compute, Storage and Reproducibility

The run uses one heavy job, two CPU threads and reduced scheduling priority. Packed real-token float16 features are padded only in small batches, then their cache is removed. Guards record resource snapshots and enforce 3 GiB free disk, 8% available memory and a 0.75 GiB swap-growth allowance.

The initial dense-cache extraction was stopped after memory pressure grew. The completed run uses a disk-backed packed cache. No unrelated process, user dataset or prior model is removed. Periodic thresholds reduce operational risk but cannot control every background operating-system change.

Pinned revisions, corpus hashes, encoder fingerprints, configurations and per-case results support reproducibility. The local launcher serves port 8501. Docker/Render configuration does not prove remote deployment: weights and generated artifacts are ignored by Git and require explicit distribution.

Fewer trainable parameters reduce adaptation and optimizer-state cost. BERT still runs at inference, and plan search adds work. Reported local latency is consequently separate from training efficiency and semantic accuracy; the new head is not claimed to be uniformly faster.

VIII. LIMITATIONS AND FUTURE WORK

The current automata task is binary. Plan training covers a finite operator vocabulary, bounded numbers and selected literal lengths. A valid plan may misunderstand negation, numeric scope, conjunction or a paraphrase outside training. The decoder can abstain on invalid output, but it has no calibrated semantic-confidence guarantee. Exact DFA construction from a plan proves implementation semantics; it does not prove that the plan is the intended interpretation of English.

The new comparison is generated and uses one training seed. Language-group protection reduces formal-target leakage, but it does not turn shared operator families into unseen mathematical concepts. The legacy V3 test has already influenced development and is now a regression benchmark. The retired mixed challenge is also not an untouched test for future iterations. Further work needs independently authored requests, family-held-out cases, numeric extrapolation, paired minimal edits and fresh test versions.

Grammar exactness depends on canonical productions; general CFG equivalence is not supplied. Compiler support covers symbolic arithmetic, without statements, optimization, machine code or runtime execution. Correct rejection counts in negative compiler cases. These are task boundaries, not general compiler accuracy.

Further work needs matched-data comparisons, several seeds, balanced supervision and calibrated abstention. An editable predicted plan could expose ambiguity while preserving the diagram. Quantization should require semantic regression checks. Independent human-authored tests and an educational user study remain necessary.

IX. CONCLUSION

The typed BERT plan improves measured correctness while preserving the visualization. Validation gates and resource logs constrain deployment. Checked formal evidence supports specific guarantees; arbitrary English interpretation remains an empirical limitation.

REFERENCES

[1] N. Kushman and R. Barzilay, "Using Semantic Unification to Generate Regular Expressions from Natural Language," NAACL-HLT, pp. 826-836, 2013. Online source.

[2] N. Locascio, K. Narasimhan, E. DeLeon, N. Kushman, and R. Barzilay, "Neural Generation of Regular Expressions from Natural Language with Minimal Domain Knowledge," EMNLP, pp. 1918-1923, 2016. Online source.

[3] A. Vaswani et al., "Attention Is All You Need," Advances in Neural Information Processing Systems, vol. 30, 2017. Online source.

[4] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," NAACL-HLT, pp. 4171-4186, 2019. Online source.

[5] Y. Liu et al., "RoBERTa: A Robustly Optimized BERT Pretraining Approach," arXiv:1907.11692, 2019. Online source.

[6] W. Tai et al., "exBERT: Extending Pre-trained Models with Domain-specific Vocabulary Under Constrained Training Resources," Findings of EMNLP, pp. 1433-1439, 2020. Online source.

[7] J. Lee et al., "BioBERT: a pre-trained biomedical language representation model for biomedical text mining," Bioinformatics, vol. 36, no. 4, pp. 1234-1240, 2020. Online source.

[8] L. J. Weber et al., "Efficient Token Augmented Fine-Tuning," IEEE CSDE, 2023. Online source.

[9] N. Locascio, DeepRegex KB13 dataset mirror, revision 096490db, MIT license. Online source.

[10] J. Herzig et al., "Unlocking Compositional Generalization in Pre-trained Models Using Intermediate Representations," arXiv:2104.07478, 2021. Online source.

[11] I. Oren, J. Herzig, N. Gupta, M. Gardner, and J. Berant, "Improving Compositional Generalization in Semantic Parsing," Findings of EMNLP, 2020. Online source.

[12] T. Scholak, N. Schucher, and D. Bahdanau, "PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models," EMNLP, pp. 9895-9901, 2021. Online source.

[13] M. T. Ribeiro, T. Wu, C. Guestrin, and S. Singh, "Beyond Accuracy: Behavioral Testing of NLP Models with CheckList," ACL, pp. 4902-4912, 2020. Online source.

[14] Cornell CS 4120, "Automating Lexical Analysis," course notes, Spring 2023. Online source.

[15] NLTK Project, nltk.parse.earleychart: Incremental chart parsing documentation. Online source.

[16] G. Deletang et al., "Neural Networks and the Chomsky Hierarchy," ICLR, 2023; arXiv:2207.02098. Online source.

[17] L. Strobl, W. Merrill, G. Weiss, D. Chiang, and D. Angluin, "What Formal Languages Can Transformers Express? A Survey," TACL, vol. 12, pp. 543-561, 2024. Online source.

[18] Google BERT / Hugging Face, bert-base-uncased model artifact and model card. Online source.

[19] TheoryBERT project repository, local implementation snapshot, 2026. Online source.